

El nacimiento del Context Engineering

Módulo 3 — Context Engineering Profesional

Introducción

Durante los primeros años de adopción de los modelos de lenguaje, la mayor parte del esfuerzo de los usuarios se concentró en aprender a escribir mejores prompts. La disciplina conocida como *Prompt Engineering* permitió descubrir que pequeños cambios en la forma de redactar una instrucción podían producir diferencias significativas en la calidad de las respuestas.

Sin embargo, a medida que los modelos evolucionaron, también lo hicieron las aplicaciones construidas sobre ellos. Los asistentes conversacionales dejaron de responder únicamente a una pregunta aislada y comenzaron a interactuar con herramientas, consultar bases de conocimiento, mantener memoria de conversaciones anteriores y ejecutar acciones sobre sistemas externos.

En ese nuevo escenario apareció un problema evidente: un excelente prompt ya no era suficiente.

Lo verdaderamente importante pasó a ser **todo el contexto** que recibe el modelo antes de generar una respuesta. El contexto puede definirse como:

Toda la información disponible para el modelo en el instante en que debe generar una respuesta.

Esta definición incluye mucho más que el mensaje del usuario: instrucciones del sistema, historial conversacional, memoria persistente, documentos recuperados mediante RAG, resultados de herramientas, datos del usuario, políticas de seguridad y restricciones del dominio.

Objetivos de esta sección

Al finalizar esta sección el lector podrá:

- Comprender por qué Prompt Engineering dejó de ser suficiente.
 - Diferenciar un prompt de una arquitectura de contexto.
 - Entender el concepto moderno de Context Engineering.
 - Identificar los componentes que forman el contexto de un LLM.
-

La evolución hacia el Context Engineering

Toda disciplina madura atraviesa un proceso de evolución. En el caso de la Ingeniería de IA, esa evolución fue especialmente rápida. Entre 2022 y 2026 el foco pasó de aprender a formular preguntas a diseñar sistemas completos capaces de proporcionar a un modelo toda la información necesaria para resolver un problema.

Primera etapa: el prompt como protagonista. Con la aparición de los primeros LLM de uso masivo, el éxito dependía principalmente de cómo se redactaba una instrucción. Los usuarios experimentaban con prompts más largos, ejemplos *few-shot*, definición de roles, cadenas de razonamiento y restricciones de formato. En esa etapa el prompt era prácticamente el único mecanismo de control.

Segunda etapa: herramientas y conocimiento externo. Pronto surgió una limitación evidente: los modelos no conocían información actualizada ni podían interactuar con sistemas externos. Para superar esa barrera aparecieron la recuperación de documentos (RAG), las llamadas a funciones (*Function Calling*), las búsquedas web y el acceso a bases de datos. El modelo comenzó a trabajar con información dinámica.

Tercera etapa: memoria y estado. Los asistentes modernos ya no resuelven una única consulta aislada. Necesitan recordar quién es el usuario, qué ocurrió anteriormente, qué tareas están en ejecución y cuáles son sus preferencias. La conversación se transforma en un proceso continuo y el contexto adquiere una dimensión temporal.

Cuarta etapa: Context Engineering. En la actualidad el desafío consiste en decidir qué información incorporar, cuál debe descartarse, cuándo recuperar conocimiento, cómo organizar la memoria, qué herramientas utilizar y qué restricciones aplicar. Estas decisiones forman parte del diseño de una arquitectura de contexto. El AI Engineer ya no optimiza únicamente un prompt: optimiza el flujo completo de información que rodea al modelo.

Definición

Context Engineering es la disciplina que diseña, organiza y administra toda la información que un modelo de IA recibe durante una interacción para maximizar la calidad, precisión y utilidad de sus respuestas.

No se trata únicamente de escribir instrucciones. Se trata de decidir:

- qué información enviar;
- cuándo enviarla;
- en qué formato;
- con qué prioridad;
- durante cuánto tiempo conservarla.

Estas decisiones constituyen problemas de arquitectura y no simplemente de redacción.

Comparación

Prompt Engineering	Context Engineering
Optimiza una instrucción	Optimiza todo el contexto
Centrado en el prompt	Centrado en el sistema
Resultado puntual	Conversaciones persistentes
Poco estado	Gestión de memoria
Sin herramientas	Integración de herramientas y datos

Caso de estudio

Imagine un asistente corporativo para un sistema de tickets.

Si únicamente recibe el mensaje:

"Mostrame los incidentes críticos."

la respuesta dependerá completamente del contexto disponible.

Si además conoce la identidad del usuario, su área de trabajo, el idioma preferido, los permisos asignados, el estado actual de los incidentes, la fecha y hora, y las políticas internas de la organización, la respuesta será considerablemente más precisa y útil.

La diferencia no la produjo un mejor prompt, sino un mejor contexto.

Ideas clave

- Los prompts siguen siendo importantes.
 - El contexto es más amplio que el prompt.
 - La arquitectura del contexto determina gran parte de la calidad de una solución basada en IA.
 - Context Engineering constituye una competencia fundamental para cualquier AI Engineer moderno.
-

Resumen

El Prompt Engineering continúa siendo una habilidad valiosa, pero ya no resulta suficiente para construir soluciones empresariales complejas. Este módulo estudia cómo diseñar arquitecturas completas de contexto capaces de alimentar a un modelo con la información adecuada en el momento correcto.

En la siguiente sección analizaremos en detalle todos los componentes que conforman el contexto de un modelo de lenguaje moderno.

Los componentes del contexto

Módulo 3 — Context Engineering Profesional

Introducción

Cuando un usuario escribe un mensaje en ChatGPT, Claude o Gemini, suele imaginar que el modelo responde únicamente a ese texto. En realidad, el modelo recibe un conjunto mucho más amplio de información.

En la sección anterior definimos el contexto como toda la información disponible para el modelo en el instante en que debe generar una respuesta. En esta sección exploraremos su estructura interna: qué elementos lo componen y cómo se relacionan entre sí.

Comprender estos componentes es el primer paso para diseñar soluciones profesionales de IA.

El contexto como una arquitectura

Una forma útil de visualizar el contexto es pensarlo como una pila de capas apiladas sobre el modelo.



Cada una de estas capas aporta información que modifica la forma en que el modelo razona.

Los componentes principales

1. Instrucciones del sistema

Definen el comportamiento esperado del modelo.

Ejemplos:

- rol profesional;
 - idioma;
 - formato de respuesta;
 - restricciones;
 - objetivos.
-

2. Mensaje del usuario

Representa la necesidad inmediata que debe resolverse.

No suele contener toda la información necesaria para producir la mejor respuesta.

3. Historial

Permite mantener continuidad entre distintas interacciones.

Gracias al historial el modelo puede interpretar referencias como:

"Hacé lo mismo que antes."

Sin historial esa instrucción carecería de significado.

4. Memoria

La memoria conserva información que debe sobrevivir a una conversación específica.

Por ejemplo:

- preferencias del usuario;
- proyectos activos;
- estilo de comunicación;
- configuraciones frecuentes.

La diferencia entre memoria e historial es fundamental: el historial describe lo que ocurrió en la conversación actual; la memoria preserva lo que el sistema sabe del usuario más allá de esa conversación. Una sesión puede terminar y la memoria continúa existiendo, lista para enriquecer la siguiente interacción.

5. Conocimiento recuperado

Muchas aplicaciones incorporan documentación obtenida dinámicamente mediante RAG.

Esto permite responder utilizando información reciente sin necesidad de reentrenar el modelo.

6. Herramientas

Los modelos modernos pueden consultar APIs, ejecutar funciones o acceder a bases de datos.

El resultado de esas operaciones también pasa a formar parte del contexto.

Error frecuente

Uno de los errores más comunes consiste en invertir todo el esfuerzo en escribir un prompt perfecto mientras se descuida el resto del contexto.

En aplicaciones empresariales suele ocurrir exactamente lo contrario: un prompt sencillo acompañado por un contexto bien diseñado produce mejores resultados que un prompt extremadamente elaborado pero aislado.

Nota del arquitecto

Antes de optimizar un prompt, pregúntese:

- ¿El modelo tiene toda la información necesaria?
- ¿Existe información irrelevante ocupando tokens?
- ¿Hay datos desactualizados?
- ¿Qué podría recuperarse dinámicamente en lugar de enviarse siempre?

Estas preguntas suelen tener un impacto mucho mayor que modificar unas pocas palabras del prompt.

Resumen

El contexto constituye el verdadero espacio de trabajo de un modelo de lenguaje. El prompt del usuario representa solo uno de sus componentes.

En la siguiente sección analizaremos cómo organizar estos componentes en una arquitectura de capas y qué principios permiten construir contextos eficientes, escalables y mantenibles.

Las capas del contexto

Módulo 3 — Context Engineering Profesional

Introducción

En la sección anterior identificamos los componentes que integran el contexto de un modelo de lenguaje. En esta sección profundizaremos en una idea fundamental para cualquier AI Engineer: **el contexto debe diseñarse como una arquitectura en capas**, no como un bloque de texto.

Esta forma de pensar permite construir soluciones más mantenibles, escalables y fáciles de evolucionar.

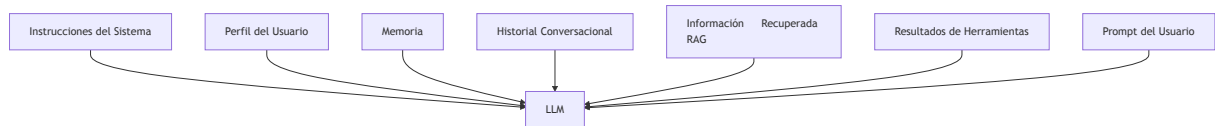
El contexto no es un prompt

Uno de los errores más frecuentes al comenzar a trabajar con LLM consiste en concentrar toda la lógica dentro del prompt del usuario.

En sistemas empresariales, esa aproximación rápidamente se vuelve inmanejable.

Un buen diseño distribuye las responsabilidades entre distintas capas, cada una con un propósito específico.

Arquitectura de capas



Cada una de estas capas puede evolucionar independientemente sin modificar las demás.

Capa 1 — Instrucciones del sistema

Es la capa más estable.

Define:

- identidad del asistente;
- objetivos;
- restricciones;
- tono;
- formato esperado;
- políticas.

Normalmente cambia muy poco durante la vida útil de una aplicación.

Capa 2 — Perfil del usuario

Contiene información relativamente estable.

Ejemplos:

- idioma;
- zona horaria;
- permisos;
- organización;
- preferencias.

No pertenece al prompt, sino al usuario.

Capa 3 — Memoria

La memoria almacena información adquirida durante el uso del sistema.

Puede incluir:

- proyectos activos;
- decisiones previas;
- preferencias aprendidas;
- tareas pendientes.

Su objetivo es evitar que el usuario deba repetir constantemente la misma información.

Capa 4 — Historial

Describe únicamente la conversación actual.

No debe confundirse con la memoria.

Una conversación puede finalizar mientras la memoria continúa existiendo.

Esta separación mejora significativamente la calidad del diseño.

Capa 5 — Información recuperada

Antes de responder, el sistema puede consultar:

- documentación;
- bases de conocimiento;
- normativa;
- manuales;
- bases de datos.

Esta información suele incorporarse mediante técnicas de recuperación (RAG).

Capa 6 — Herramientas

Muchas respuestas requieren ejecutar acciones.

Por ejemplo:

- consultar una API;
- crear un ticket;
- enviar un correo;
- obtener información meteorológica;
- ejecutar una consulta SQL.

Los resultados de esas acciones también forman parte del contexto.

Principio de responsabilidad única

Cada capa debería responder una única pregunta:

Capa	Pregunta
Sistema	¿Cómo debe comportarse el modelo?
Usuario	¿Quién realiza la consulta?
Memoria	¿Qué recuerda el sistema?
Historial	¿Qué ocurrió en esta conversación?
Recuperación	¿Qué conocimiento necesito ahora?
Herramientas	¿Qué información debo obtener o qué acción debo ejecutar?

Cuando una capa intenta resolver responsabilidades de otra, la arquitectura comienza a degradarse.

Nota del arquitecto

Una buena práctica consiste en poder modificar cualquiera de las capas sin afectar al resto del sistema.

Si cambiar el idioma del usuario obliga a modificar el prompt principal, probablemente exista un problema de diseño.

Resumen

El Context Engineering no consiste únicamente en enviar más información al modelo. Consiste en organizar esa información de forma coherente.

Pensar en capas facilita la reutilización, reduce errores y prepara el camino para arquitecturas más avanzadas como RAG, agentes y sistemas multiagente.

En la próxima sección estudiaremos cómo un modelo procesa internamente ese contexto y por qué el orden y la estructura de la información influyen directamente en la calidad de las respuestas.

Cómo procesa un LLM el contexto

Módulo 3 — Context Engineering Profesional

Introducción

Hasta aquí vimos qué es el contexto y cómo organizarlo en capas. Sin embargo, todavía falta responder una pregunta fundamental:

¿Qué hace realmente un modelo de lenguaje con toda esa información?

Responder esta pregunta permite comprender por qué algunas arquitecturas producen resultados consistentes mientras que otras fallan incluso utilizando el mismo modelo.

El contexto como espacio de trabajo

Un LLM no "consulta una base de datos interna" cada vez que responde. En cada interacción construye un espacio de trabajo temporal compuesto por todos los tokens que recibe.

En ese espacio conviven simultáneamente:

- las instrucciones del sistema;
- el historial de la conversación;
- el mensaje del usuario;
- la memoria incorporada;
- los documentos recuperados;
- los resultados de herramientas.

La respuesta se genera utilizando únicamente la información disponible en ese instante.

Un modelo no distingue el origen de los datos

Desde el punto de vista del modelo, toda la información llega convertida en tokens.

Eso significa que una regla del sistema, un párrafo recuperado mediante RAG o el mensaje escrito por el usuario terminan formando parte de la misma secuencia de entrada.

La responsabilidad de priorizar y estructurar esa información recae en el diseñador de la solución.

El orden importa

Aunque los modelos modernos son muy potentes, el orden en que se presenta la información influye en el resultado. Los mecanismos de atención que sustentan a los transformadores asignan mayor peso a la información más cercana al final de la secuencia; por eso las instrucciones críticas y la consulta del usuario tienden a posicionarse en las últimas posiciones.

Una organización habitual es:

1. Instrucciones del sistema.
2. Información permanente.
3. Memoria.
4. Historial.
5. Conocimiento recuperado.
6. Consulta del usuario.

Este orden facilita que el modelo interprete correctamente las prioridades.

Señales contradictorias

Uno de los problemas más comunes ocurre cuando distintas partes del contexto contienen instrucciones incompatibles.

Ejemplo:

- El sistema indica responder siempre en español.
- Un documento recuperado contiene instrucciones para responder en inglés.
- El usuario solicita un resumen en francés.

Sin una jerarquía clara, el comportamiento puede ser impredecible.

Las arquitecturas modernas resuelven este problema estableciendo reglas explícitas de precedencia. Una jerarquía de referencia sencilla y efectiva es la siguiente:

1. **Instrucciones del sistema** (máxima prioridad). Definen el comportamiento base de la aplicación.
2. **Políticas y restricciones** (prioridad alta). Reglas de negocio, seguridad y cumplimiento que no pueden ser sobrescritas por el usuario.
3. **Solicitud del usuario** (prioridad normal). La instrucción del usuario rige en todo lo que no entre en conflicto con los niveles superiores.

Volviendo al ejemplo anterior: el sistema indica español, el documento recuperado incluye texto en inglés (que el modelo puede usar como fuente pero no como instrucción de idioma) y el usuario pide francés. Aplicando la jerarquía, el modelo responde en español porque la instrucción del sistema tiene precedencia sobre la solicitud del usuario. El diseñador puede optar por permitir excepciones de idioma si así lo especifica explícitamente en la capa de sistema.

Caso práctico

Supongamos un asistente jurídico.

Antes de responder una consulta podría recibir:

- políticas institucionales;
- normativa vigente;
- fallos judiciales relevantes;
- historial del expediente;
- perfil del abogado que realiza la consulta;
- pregunta del usuario.

El valor del sistema no depende únicamente del modelo utilizado, sino de cómo se construyó ese contexto.

Error frecuente

Agregar más información no siempre mejora la respuesta.

Un contexto excesivamente grande puede introducir ruido, aumentar el costo y dificultar que el modelo identifique los elementos realmente importantes.

En Context Engineering, **calidad** suele ser más importante que **cantidad**.

Resumen

Un LLM procesa el contexto como una única secuencia de información disponible durante la inferencia. El diseñador de la solución debe decidir qué datos incluir, en qué orden presentarlos y cuáles descartar. Establecer jerarquías de precedencia entre capas es la herramienta clave para resolver conflictos y garantizar un comportamiento predecible.

En la siguiente sección estudiaremos los principios de diseño que utilizan las arquitecturas modernas para construir contextos robustos y eficientes.

Principios de diseño del contexto

Módulo 3 — Context Engineering Profesional

Introducción

Diseñar el contexto de una aplicación basada en LLM no consiste en agregar toda la información disponible. El objetivo es proporcionar **la información correcta, en el momento correcto y con la estructura adecuada.**

En esta sección analizaremos los principios que sirven como guía para construir arquitecturas de contexto eficientes.

Principio 1 — Relevancia

Todo elemento incorporado al contexto debe contribuir a resolver la tarea.

Antes de agregar información, pregúntese:

- ¿Es necesaria para responder?
- ¿Aporta valor?
- ¿Puede recuperarse bajo demanda?

La información irrelevante incrementa el consumo de tokens y dificulta el razonamiento.

Principio 2 — Jerarquía

No toda la información tiene la misma importancia.

Una jerarquía habitual es:

1. Instrucciones del sistema.
2. Políticas y restricciones.
3. Memoria persistente.
4. Historial.
5. Información recuperada.
6. Consulta del usuario.

Cuando existen conflictos, esta jerarquía permite resolverlos de manera consistente.

Principio 3 — Modularidad

Cada componente del contexto debería mantenerse de forma independiente.

Ejemplos:

- perfil del usuario;
- memoria;
- documentos RAG;
- herramientas;
- historial.

La modularidad simplifica el mantenimiento y favorece la reutilización. Si cambiar el idioma del usuario exige modificar el prompt principal, la modularidad está comprometida.

Principio 4 — Actualización

El contexto debe reflejar el estado actual del sistema.

Es recomendable:

- eliminar información obsoleta;
 - refrescar datos dinámicos;
 - invalidar memorias incorrectas;
 - volver a consultar fuentes cuando sea necesario.
-

Principio 5 — Economía de tokens

Los modelos tienen un límite de contexto y cada token tiene un costo.

Una arquitectura eficiente:

- resume información repetitiva;
 - evita duplicados;
 - utiliza RAG para recuperar únicamente lo necesario;
 - elimina contenido que ya no aporta valor.
-

Ejemplo

Un asistente de soporte recibe la solicitud:

"¿Cuál es el estado del incidente 1542?"

En lugar de enviar toda la base de tickets al modelo, el sistema:

1. identifica el incidente;
2. recupera únicamente su información;
3. incorpora el perfil del usuario;
4. agrega las políticas de respuesta;
5. envía el contexto al LLM.

El resultado es una respuesta más rápida, económica y precisa.

Checklist de diseño

Antes de poner en producción una solución, verifique:

- ¿Cada bloque del contexto tiene un propósito?
 - ¿Existe información redundante?
 - ¿Las prioridades están definidas?
 - ¿La memoria puede actualizarse?
 - ¿El conocimiento recuperado está vigente?
 - ¿El modelo tiene toda la información necesaria para responder?
 - ¿Hay datos desactualizados que podrían inducir a error?
 - ¿Existe información que podría recuperarse bajo demanda en lugar de enviarse siempre?
-

Resumen

El Context Engineering combina principios de arquitectura de software con técnicas específicas para modelos de lenguaje. La calidad del contexto depende menos de su tamaño que de su organización.

En la próxima sección pondremos en práctica estos principios mediante un laboratorio rápido de diseño de arquitectura de contexto.

Laboratorio rápido: diseñar una arquitectura de contexto

Módulo 3 — Context Engineering Profesional

Introducción

Los principios del Context Engineering se comprenden mejor cuando se aplican a un problema real. Este laboratorio propone un ejercicio estructurado de diseño que puede completarse en aproximadamente diez minutos.

El objetivo no es producir una implementación funcional, sino entrenar la capacidad de pensar en términos de capas de contexto antes de escribir una sola línea de código.

Enunciado

Elija un asistente de IA que podría ser útil en su dominio de trabajo. Puede ser real o hipotético. Algunos ejemplos orientativos:

- un asistente para un equipo de desarrollo de software;
 - un asistente jurídico para un estudio de abogados;
 - un asistente de onboarding para nuevos empleados;
 - un asistente de análisis de datos para un equipo de finanzas;
 - un asistente de soporte técnico para un producto SaaS.
-

Paso 1 — Definir el asistente

Responda brevemente:

- ¿Qué problema resuelve?
 - ¿Quiénes son sus usuarios?
 - ¿Cuál es la consulta más frecuente que recibirá?
-

Paso 2 — Completar el esquema de capas

Para el asistente elegido, identifique qué información debería colocarse en cada capa del contexto. Use la siguiente plantilla:

INSTRUCCIONES DEL SISTEMA

Rol, objetivos, restricciones, tono, formato de salida

→

PERFIL DEL USUARIO

Datos estables sobre quien realiza la consulta

→

MEMORIA

Información persistente entre conversaciones

→

INFORMACIÓN RECUPERADA (RAG)

Documentos o datos obtenidos dinámicamente

→

HERRAMIENTAS Y APIs

Acciones o fuentes de datos en tiempo real

→

| POLÍTICAS Y RESTRICCIONES |

| Reglas de negocio, seguridad y cumplimiento |

|

| → |

Paso 3 — Identificar decisiones de diseño

Una vez completado el esquema, responda:

1. ¿Qué capa tiene mayor impacto en la calidad de las respuestas para este caso de uso?
 2. ¿Qué información podría recuperarse bajo demanda en lugar de enviarse siempre?
 3. ¿Existe alguna capa vacía? ¿Es intencional o es un problema de diseño?
 4. ¿Cómo resolverá el sistema un conflicto entre lo que pide el usuario y una política definida en el sistema?
-

Ejemplo resuelto: asistente de soporte técnico

Descripción: asistente interno para un equipo de soporte de primer nivel de una empresa de software.

Capa de sistema: "Eres un agente de soporte técnico para [Producto]. Responde en el idioma del usuario. No compartas información de otros clientes. Escala los incidentes P1 al equipo de guardia de forma inmediata."

Perfil del usuario: nombre, área, nivel de acceso al sistema, tickets previos abiertos en los últimos 30 días.

Memoria: preferencias de comunicación del usuario (técnica vs. simplificada), historial de incidentes recurrentes.

RAG: base de conocimiento de soluciones conocidas, release notes de las últimas versiones, guías de troubleshooting.

Herramientas: API del sistema de tickets (consultar y actualizar estado), API de monitoreo (ver estado de servicios), directorio de contactos del equipo de guardia.

Políticas: los incidentes P1 no pueden cerrarse sin aprobación del equipo de ingeniería; no se comparte información de configuración de producción con usuarios sin acceso de nivel 3.

Decisiones de diseño:

- El estado del incidente se recupera en tiempo real (herramienta), no se precarga en memoria.
 - Las políticas de escalamiento tienen precedencia sobre cualquier instrucción del usuario.
 - La base de conocimiento se filtra por versión del producto antes de enviarse al modelo.
-

Cierre del laboratorio

No existe una única respuesta correcta. La validez de un diseño de contexto se mide por su capacidad para producir respuestas precisas, predecibles y económicas a lo largo del tiempo.

En la siguiente sección integraremos todos estos conceptos mediante un caso de estudio completo que muestra la evolución de una solución real, desde un prompt inicial hasta una arquitectura profesional.

Caso de estudio: de un prompt a una arquitectura de contexto

Módulo 3 — Context Engineering Profesional

Introducción

Hasta este punto estudiamos los fundamentos del Context Engineering. Ahora integraremos esos conceptos mediante un caso práctico que muestra cómo evoluciona una solución desde un simple prompt hasta una arquitectura completa de contexto.

Escenario

Una organización desea desarrollar un asistente interno para su mesa de ayuda de TI.

Los usuarios esperan poder:

- consultar el estado de incidentes;
- buscar documentación técnica;
- solicitar nuevas altas;
- conocer el estado de cambios programados.

La primera implementación consiste únicamente en un prompt.

```
Actuá como un agente de soporte técnico y respondé las preguntas del usuario.
```

Aunque funciona para demostraciones, rápidamente aparecen limitaciones.

Problemas detectados

- El asistente desconoce quién realiza la consulta.
- No tiene acceso al sistema de tickets.
- No recuerda conversaciones anteriores.
- No consulta la documentación vigente.
- Puede responder utilizando información desactualizada.

En consecuencia, las respuestas resultan inconsistentes.

Evolución de la solución

La aplicación incorpora progresivamente nuevas capas de contexto.

Paso 1 — Instrucciones del sistema

Se definen el rol, el idioma, el formato de salida y las políticas generales.

Paso 2 — Perfil del usuario

Antes de consultar al modelo se agregan:

- nombre;
- área;
- permisos;
- idioma;
- zona horaria.

Paso 3 — Recuperación de conocimiento

El sistema consulta la base documental mediante RAG y envía únicamente los documentos relevantes.

Paso 4 — Herramientas

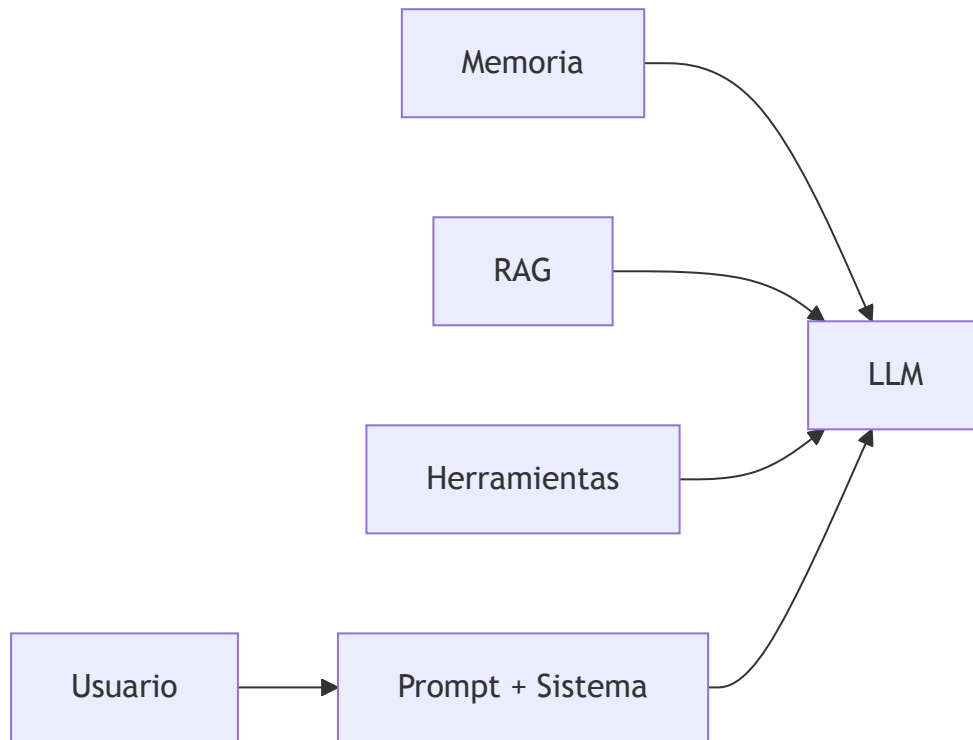
El modelo obtiene información actualizada desde:

- API de tickets;
- CMDB;
- inventario;
- calendario de mantenimiento.

Paso 5 — Memoria

Se incorporan preferencias y decisiones persistentes del usuario para evitar repetir información en cada interacción.

Arquitectura resultante



Beneficios obtenidos

La nueva arquitectura permite:

- respuestas más precisas;
 - menor cantidad de alucinaciones;
 - continuidad entre conversaciones;
 - acceso a información actualizada;
 - menor cantidad de tokens enviados de forma innecesaria.
-

Lecciones aprendidas

1. El prompt sigue siendo importante, pero ya no es el centro de la solución.
 2. La calidad depende de la arquitectura del contexto.
 3. Cada componente debe tener una responsabilidad clara.
 4. El contexto debe mantenerse actualizado durante todo el ciclo de vida de la aplicación.
-

Resumen

El Context Engineering transforma un asistente basado únicamente en prompts en una solución empresarial capaz de combinar memoria, conocimiento, herramientas y reglas de negocio. Esta evolución constituye uno de los cambios más importantes en la Ingeniería de IA moderna.

La siguiente y última sección del capítulo resumirá los conceptos principales y presentará una guía de autoevaluación para el lector.

Cierre del capítulo y autoevaluación

Módulo 3 — Context Engineering Profesional

Introducción

En este primer capítulo presentamos los fundamentos del Context Engineering y explicamos por qué esta disciplina se ha convertido en una pieza central de la Ingeniería de IA moderna.

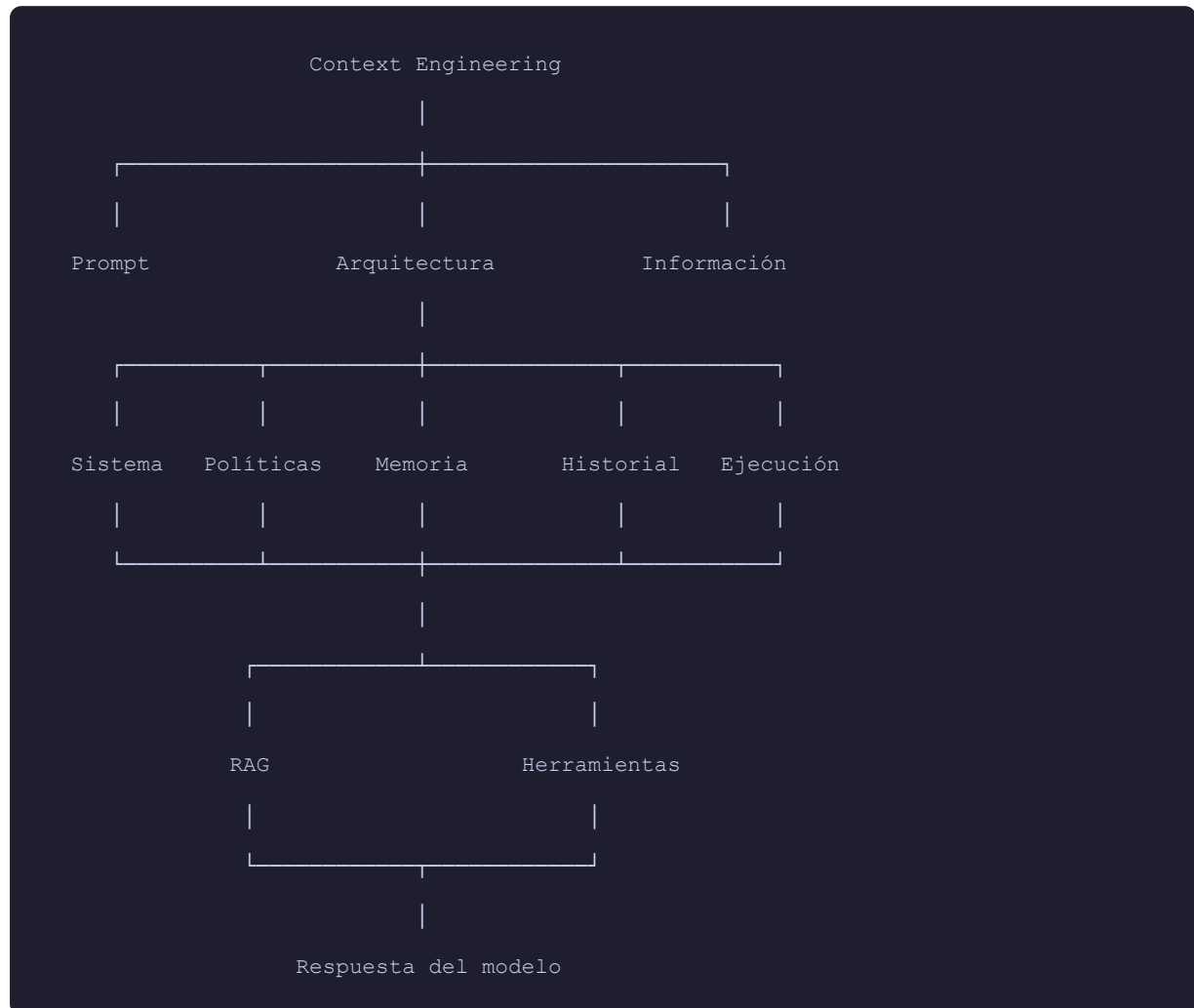
El objetivo no fue enseñar una herramienta específica, sino desarrollar una nueva forma de pensar el diseño de soluciones basadas en modelos de lenguaje.

Ideas clave

Al finalizar este capítulo deberías poder explicar con tus propias palabras que:

- un prompt no representa todo el contexto;
 - el contexto está compuesto por múltiples capas;
 - la calidad de una solución depende de la arquitectura del contexto;
 - memoria, RAG y herramientas forman parte del contexto;
 - agregar más información no siempre produce mejores respuestas;
 - las jerarquías de precedencia permiten resolver conflictos entre capas de manera predecible.
-

Mapa conceptual



Preguntas de autoevaluación

Intenta responder sin consultar el material.

1. ¿Cuál es la diferencia entre Prompt Engineering y Context Engineering?
2. ¿Qué elementos forman parte del contexto de un LLM?
3. ¿Cuál es la diferencia entre memoria e historial?
4. ¿Por qué RAG forma parte del contexto?
5. ¿Qué riesgos presenta un contexto excesivamente grande?
6. ¿Por qué resulta importante definir una jerarquía entre las distintas capas?
7. ¿Cómo se resuelve un conflicto entre una instrucción del sistema y una solicitud del usuario?

Si puedes responder con claridad estas preguntas, has comprendido los conceptos esenciales del capítulo.

Ejercicio práctico

Selecciona una aplicación basada en IA que utilices habitualmente.

Analiza:

- qué información recibe el modelo;
- qué datos parecen provenir de memoria;
- qué información podría obtener mediante RAG;
- qué herramientas podrían intervenir;
- qué políticas o restricciones parecen estar presentes;
- qué mejorarías en la arquitectura del contexto.

No busques la respuesta "correcta". El objetivo es entrenar la capacidad de analizar sistemas desde la perspectiva del Context Engineering.

Errores frecuentes

Durante los primeros proyectos es habitual encontrar algunos patrones:

- concentrar toda la lógica en el prompt;
- duplicar información en distintas capas;
- mantener documentos desactualizados;
- enviar más contexto del necesario;
- no diferenciar memoria de historial;
- omitir las políticas de precedencia ante conflictos entre capas.

Detectar estos problemas temprano simplifica enormemente la evolución del sistema.

Lo que viene

En el próximo capítulo estudiaremos la anatomía del contexto con mayor profundidad.

Analizaremos:

- el ciclo de vida del contexto;
- la gestión de tokens;
- los límites de las ventanas de contexto;
- estrategias de compresión;
- técnicas para construir contextos dinámicos.

Estos conceptos servirán de base para el resto del módulo y para el diseño de soluciones empresariales de IA.

Conclusión

El Context Engineering representa un cambio de paradigma. La calidad de una solución ya no depende únicamente del modelo utilizado ni de la redacción de un prompt, sino del diseño completo del flujo de información que acompaña al modelo durante cada interacción.

Dominar esta disciplina permitirá construir asistentes, agentes y aplicaciones mucho más robustos, explicables y escalables.